

Pontificia Universidade Catolica - Minas Gerais

Ciência da Computação

Andriel Mark da Silva Pinto

Beatriz Miranda Paiva

Felipe Henrique Oliveira Diniz

Matheus de Almeida Moreira

Otoniel Goulart Damasceno Filho

Relatório de Desenvolvimento: Controlador de Cache L1

Belo Horizonte

2026

1. Introdução

A hierarquia de memória é um conceito fundamental na arquitetura de computadores modernos, mitigando a discrepância de velocidade entre a CPU e a memória principal. O uso de memórias cache L1 permite o armazenamento temporário das informações mais acessadas, reduzindo a latência através do princípio da localidade (espacial e temporal). Este relatório detalha o desenvolvimento, em SystemVerilog, de um controlador de cache associativo por conjuntos de 2 vias. O projeto explora o ciclo completo de tratamento de *hits* e *misses*, validação de políticas de atualização e consistência de dados entre níveis de memória.

2. Descrição do Desenvolvimento

2.1. Arquitetura do Controlador:

O sistema foi concebido para um barramento de endereços de 8 bits e dados de 32 bits. A decomposição do endereço foi estruturada da seguinte forma:

- **Tag:** 4 bits ([7:4]), garantindo a identificação única do bloco.
- **Index:** 2 bits ([3:2]), selecionando um dos 4 conjuntos ($\log_2(4) = 2$).
- **Byte Offset:** 2 bits ([1:0]), ignorados pela lógica de controle pois o sistema opera em palavras inteiras de 32 bits.

O coração do controlador é uma Máquina de Estados Finitos (FSM) de 8 estados:

- "ST_IDLE": Aguarda as requisições da CPU.
- "ST_COMPARE": Realiza a comparação simultânea das tags das duas vias e atualiza o LRU em caso de *hit*.
- "ST_WB_REQ" e "ST_WB_WAIT": Tratam a expulsão de blocos modificados (*dirty*) para a memória principal.
- "ST_FETCH_REQ" e "ST_FETCH_WAIT": Buscam novos blocos na memória em caso de *miss*.
- "ST_UPDATE": Atualiza os arrays internos da cache com os dados lidos e reseta as flags.
- "ST_FINISH": Sinaliza à CPU que o dado está pronto.

2.2. Decisões de Projeto e Hipóteses

- **Design Monolítico vs Modular:** Optou-se por uma arquitetura monolítica no arquivo "cache_controller.sv". A separação das memórias de tag e dados em módulos distintos geraria um *overhead* de interface (cerca de 370 linhas contra 254 do modelo atual) sem ganho prático para uma cache didática de 8 linhas.
- **Políticas Adotadas:** Implementou-se o modelo *Write-Back* associado ao *Write-Allocate*. Essa abordagem é mais realista e eficiente no tráfego de barramento, exigindo o controle rigoroso de um *dirty bit*. A substituição de blocos segue o algoritmo *Least Recently Used* (LRU), implementado de forma econômica através de 1 bit por conjunto (indicando qual via é a mais antiga).
- **Reset:** Foi adotado um *reset* assíncrono para garantir que todas as estruturas de dados ("valid", "dirty", "tags") comecem em um estado conhecido e seguro.

2.3. Dificuldades Encontradas

O principal desafio técnico foi o sincronismo do *datapath* com os tempos de latência da memória. Foi necessário embutir rotinas de barreira ("wait(cpu_ready == 1'b1)") nos testbenches para garantir que a CPU simulada respeitasse a latência dos estados "FETCH_WAIT" e "WB_WAIT". Além disso, a manipulação simultânea do bit LRU no estado de "ST_COMPARE" (para *hits*) e no estado de "ST_UPDATE" (para *misses*) demandou cuidado para não gerar condições de corrida no RTL.

3. Resultados Obtidos

3.1. Simulações Realizadas

A validação foi centralizada em um ambiente único ("tb_cache_top.sv"), isolando os cenários através de *tasks* no SystemVerilog. Os testes cobriram:

- Comportamento puro de Read/Write Path (*Hits*).
- Injeção de múltiplos *Misses* com alocação correta.
- Forçamento de colisão no mesmo Índice para engatilhar o algoritmo LRU.
- Comprovação da transição para a rotina de *Write-Back* ao ejetar um bloco *dirty*.

3.2. Validação Funcional

A verificação foi atestada por dois métodos complementares:

- **Automação de Asserções:** Criação da macro "check_assert(nome, condicao)", gerando um relatório em log de "[PASS]" e "[FAIL]" que não exige inspeção visual constante durante o desenvolvimento incremental.
- **Inspeção de Waveforms:** Utilizou-se as diretivas "\$dumpfile" e "\$dumpvars" para exportar arquivos ".vcd", que foram analisados via GTKWave para comprovar o acionamento de "mem_write" especificamente na expulsão da via saturada.

3.3. Análise dos Resultados

O sistema cumpriu integralmente os requisitos. Observou-se, no log de testes de substituição, que o preenchimento de endereços distintos mapeados para o Índice 0 provocou, corretamente, a alocação nas Vias 0 e 1. A escrita subsequente forçou a Via 0 ao estado *dirty*, e a injeção do terceiro endereço concorrente causou a transição esperada da FSM para os estados de "ST_WB_REQ", confirmando a consistência dos algoritmos LRU e *Write-Back* em conjunto.

4. Conclusão

O desenvolvimento do controlador validou na prática o comportamento teórico de hierarquias de memória. A escolha do design monolítico facilitou o roteamento dos sinais de controle internos, enquanto a forte tipagem e uso de literais do SystemVerilog mitigou falhas lógicas na FSM. O projeto atingiu 100% de precisão nos testes criados, comprovando que as políticas de alocação e escrita foram implementadas de maneira robusta.

5. Uso de Inteligência Artificial

A inteligência artificial (Modelo Claude Opus 4.6 - Antigravity CLI) foi empregada ativamente durante o projeto utilizando a metodologia SDD (*Spec-Driven Development*). A IA não substituiu as decisões de engenharia, mas atuou como ferramenta de refatoração, automação e consulta arquitetural.

5.1. Como a IA foi utilizada

- **Geração de Infraestrutura:** Criação autônoma do "Makefile", scripts em Bash ("run_sim.sh") e arquivos ".gitignore".
- **Explicações Técnicas:** Forneceu bases visuais e textuais do particionamento de endereços e estados críticos da FSM.
- **Refatoração de Código e Testes:** Transformou os scripts fragmentados em um testbench unificado hierárquico com macros de asserção embutidas. A IA auxiliou a remover comentários de sintaxe para aplicar padrões de *Clean Code* focados na intenção do hardware.
- **Depuração de Estratégia:** Auxiliou a comprovar matematicamente e logicamente que a sequência de endereços gerada pelo grupo para testar o *Write-Back* com LRU iria, de fato, gerar os conflitos desejados no simulador.

5.2. Histórico de Prompts Utilizados

- **Prompt 1:** "Analise o enunciado do trabalho e me ajude a estruturar o repositório do projeto. Crie issues no GitHub..."
- **Prompt 2:** "Não precisamos de uma issue separada para o setup... O ideal é cadastrar as issues em ordem cronológica de execução."
- **Prompt 3:** "Na issue #3, a política de escrita foi definida como write-back, porém essa deveria ser uma decisão do grupo. Corrija..."
- **Prompt 4:** "Explique como funciona a decomposição do endereço em uma cache set-associative. Quero entender a divisão de bits..."
- **Prompt 5:** "Explique o fluxo de estados da FSM do controlador de cache. Quais são os estados e situações de transição?"
- **Prompt 6:** "As issues #1 e #2 propõem criar módulos separados... Não seria melhor adotar uma abordagem monolítica?"
- **Prompt 7:** "Implemente o cache_controller.sv seguindo a abordagem monolítica que decidimos..."
- **Prompt 8:** "Como implementar um mecanismo elegante para automatizar os logs de PASS/FAIL nos testes?"
- **Prompt 9 (Script):** "Crie sim/Makefile e sim/run_sim.sh com suporte a variáveis, argumentos de testbench e limpeza..."
- **Prompt 10 (Validação):** "Confirmação da sequência de estímulos criada pelo grupo para forçar o LRU e o write-back..."
- **Prompt 11:** "Testbench unificado pronto, mas os comentários poluem o código. Criar separações claras e substituir traduções literais..."
- **Prompt 12:** "Com base no controlador de cache parametrizado... implemente um arquivo de testbench unificado tb_cache_top.sv com macros de asserção."
- **Prompt 13:** "Refatore os comentários de dentro das tasks de teste para focar exclusivamente na intenção da engenharia de teste."

- **Prompt 14:** "Solicitação de revisão da formatação da versão final do README.md e do rascunho do relatório acadêmico previamente redigidos pelo grupo. O pedido estipula a padronização de nomes de arquivos e estados da FSM."

- **Aviso:** Para ter acesso aos prompts e suas respectivas respostas na íntegra, basta acessar o arquivo de registro correspondente disponibilizado dentro do repositório do projeto.
- **Link do Repositorio do Github:** <https://github.com/matheusmra/TP01-ARQ3>